

# Fenwick Tree / Binary Indexed Tree (BIT)

---

## 1. Overview

The Fenwick Tree, also called a Binary Indexed Tree (BIT), is a data structure that maintains an array of numbers and supports two operations efficiently:

- **Prefix sum query** – sum of elements from index 1 to  $i$
- **Point update** – add a value to an element at index  $i$

Both operations run in  $O(\log n)$  time. It uses  $O(n)$  space.

A Fenwick Tree is essentially a cleverly constructed array where each index stores the sum of a specific range of elements. The ranges are determined by the least significant set bit (LSB) of the index. This design allows prefix sums and point updates to be computed by walking up or down the tree using the LSB.

It is simpler, faster, and uses less memory than a Segment Tree for problems that only need prefix/range sums and point updates.

---

## 2. Intuition

### Simple Explanation

Imagine you have an array of numbers. You want to:

1. Find the sum of the first  $i$  elements quickly.
2. Add a value to an element at position  $i$  and keep the structure updated.

A naive approach would be  $O(n)$  for sum queries or  $O(1)$  for updates (or vice versa). A Fenwick Tree balances both to  $O(\log n)$ .

### The Core Trick

Every index  $i$  in the BIT is **responsible** for a range of elements in the original array. The range length is  $2^k$  where  $k$  is the position of the lowest set bit in  $i$ .

For example:

| Index (1-based) | Binary | Lowest Set Bit | Range Covered |
|-----------------|--------|----------------|---------------|
| 1               | 1      | 1              | [1, 1]        |
| 2               | 10     | 2              | [1, 2]        |
| 3               | 11     | 1              | [3, 3]        |
| 4               | 100    | 4              | [1, 4]        |
| 5               | 101    | 1              | [5, 5]        |
| 6               | 110    | 2              | [5, 6]        |

| Index (1-based) | Binary | Lowest Set Bit | Range Covered |
|-----------------|--------|----------------|---------------|
| 7               | 111    | 1              | [7, 7]        |
| 8               | 1000   | 8              | [1, 8]        |

## The LSB Operation

The key operation is  $i \& -i$  which isolates the lowest set bit.

- $i = 6$  (110)  $\rightarrow i \& -i = 2$  (010)
- $i = 7$  (111)  $\rightarrow i \& -i = 1$  (001)
- $i = 8$  (1000)  $\rightarrow i \& -i = 8$  (1000)

## Moving Through the Tree

- **To get prefix sum:** Subtract the LSB repeatedly ( $i -= i \& -i$ ) to move left/up and accumulate.
- **To point update:** Add the LSB repeatedly ( $i += i \& -i$ ) to move right/up and update all ranges that cover this index.

## Analogy

Think of the BIT as a hierarchy of managers:

- Index 1 manages only itself
- Index 2 manages indexes 1-2
- Index 4 manages indexes 1-4
- Index 6 manages indexes 5-6
- Index 8 manages everything (1-8)

When you update index 3, you tell manager 3, then manager 4, then manager 8, and so on — all managers who need to know about changes to index 3.

When you want the sum up to index 7, you ask manager 7, then manager 6, then manager 4 — whose ranges together cover [1,7].

## 3. When to Use It

Use a Fenwick Tree when:

| Situation                                       | Trigger Phrases                                         |
|-------------------------------------------------|---------------------------------------------------------|
| You need prefix sums with point updates         | "prefix sum", "cumulative sum", "range sum query"       |
| You need range sum queries (l to r)             | "sum from L to R", "query range"                        |
| You need to count inversions                    | "inversions", "count pairs $i < j$ with $a[i] > a[j]$ " |
| You need to update a point and query a range    | "point update, range query"                             |
| You need frequency counts with order statistics | "kth element", "order statistic", "rank"                |

| Situation                                             | Trigger Phrases                             |
|-------------------------------------------------------|---------------------------------------------|
| You need to compress coordinates and then query       | "coordinate compression", "offline queries" |
| You need 2D prefix queries (with 2D BIT)              | "2D grid sum", "matrix sum query"           |
| You need range update and range query (with two BITs) | "range add, range sum", "increment range"   |

## Common Problem Patterns

- "Given an array, answer sum queries with point updates"
- "Count number of pairs (i,j) with  $i < j$  and  $a[i] > a[j]$ "
- "Find the kth smallest element in a multiset"
- "Answer sum queries on a 2D grid with point updates"
- "Range add + range sum queries"

## 4. When Not to Use It

| Situation                                                               | Why Not                                                                                               | Better Alternative                 |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------|
| You need <b>range minimum/maximum/GCD queries</b>                       | BIT only works for <b>invertible</b> operations (like sum, xor, product). Min/max are not invertible. | Segment Tree                       |
| You need <b>range update</b> but <b>only point query</b>                | BIT works but a difference array is simpler                                                           | Difference Array                   |
| The array is static (no updates)                                        | $O(n)$ preprocessing + $O(1)$ prefix sum is better                                                    | Prefix Sum Array                   |
| You need <b>range update and range query</b>                            | Requires two BITs — a Segment Tree might be simpler to reason about                                   | Segment Tree with lazy propagation |
| You need to query arbitrary subarrays (not just prefix)                 | BIT can do range sum = $\text{prefix}(R) - \text{prefix}(L-1)$ , but only for sum-like operations     | Segment Tree                       |
| Input size is small ( $n \leq 1000$ )                                   | $O(n)$ per query is fine                                                                              | Brute force                        |
| You need to <b>insert/delete</b> elements at arbitrary positions        | BIT works on a fixed array                                                                            | Balanced BST, Treap                |
| You need <b>non-commutative</b> operations (like matrix multiplication) | BIT requires commutative operations                                                                   | Segment Tree                       |
| 0-based indexing is mandatory                                           | BIT is naturally 1-based. You can adapt, but it adds complexity.                                      | Segment Tree                       |

## Common Wrong Assumptions

- ✗ "BIT can answer range minimum queries" — No, min is not invertible.

- ❌ "BIT is always faster than Segment Tree" — For non-sum operations, you can't use BIT at all.
- ❌ "BIT works with 0-based indexing naturally" — It's much cleaner with 1-based indexing.
- ❌ "BIT can handle range updates trivially" — Range update + point query is easy, but range update + range query needs two BITs.

## 5. Core Concepts

### 5.1. 1-Based Indexing

Fenwick Tree is naturally 1-indexed. Index 0 is unused (or stores 0). When converting from 0-based input, add 1 to all indices.

**Why:** The LSB trick ( $i \& -i$ ) works correctly for 1-based indices. At index 0,  $i \& -i$  is 0, causing an infinite loop.

### 5.2. Least Significant Bit (LSB)

The core operation that makes BIT work.

```
lsb(i) = i & -i
```

- $i \& -i$  isolates the lowest set bit in binary representation.
- In two's complement,  $-i = \sim i + 1$ . So  $i \& -i$  gives the power of 2 corresponding to the lowest set bit.

**Example:**

```
i = 6 (110)
-i = -6 (010 in two's complement, assuming 3 bits)
i & -i = 010 = 2
```

### 5.3. The BIT Array

An array `bit[]` of size  $n+1$  (1-indexed). Each `bit[i]` stores the sum of a range of the original array.

The range stored at `bit[i]` is:

```
start = i - lsb(i) + 1
end   = i
```

So `bit[i]` = sum of `arr[start ... end]`.

### 5.4. Prefix Sum Query

To get sum of `arr[1..i]`:

```
sum = 0
while i > 0:
    sum += bit[i]
    i -= lsb(i)
```

This walks "up and left" through the tree, accumulating ranges.

## 5.5. Point Update

To add `delta` to `arr[i]`:

```
while i <= n:
    bit[i] += delta
    i += lsb(i)
```

This walks "up and right" through the tree, updating all ranges that include `i`.

## 5.6. Range Sum Query

Sum of `arr[1..r]`:

```
range_sum(l, r) = prefix_sum(r) - prefix_sum(l-1)
```

## 5.7. Building the BIT

**Method 1:  $O(n \log n)$**  — For each element, call point update.

**Method 2:  $O(n)$**  — Build from the array directly:

```
for i = 1 to n:
    bit[i] += arr[i]
    j = i + lsb(i)
    if j <= n:
        bit[j] += bit[i]
```

---

# 6. Step-by-Step Algorithm

## 6.1. Build BIT from Array

**Input:** Array `arr[1..n]` (1-indexed)

1. Initialize `bit[1..n] = 0`
2. For `i = 1` to `n`:

- Set `bit[i] += arr[i]`
- Set `j = i + (i & -i)`
- If `j <= n`, set `bit[j] += bit[i]`

## 6.2. Point Update

**Input:** Index `i`, delta `delta`

1. While `i <= n`:
  - `bit[i] += delta`
  - `i += i & -i`

## 6.3. Prefix Sum Query

**Input:** Index `i`

1. Set `sum = 0`
2. While `i > 0`:
  - `sum += bit[i]`
  - `i -= i & -i`
3. Return `sum`

## 6.4. Range Sum Query

**Input:** Range `[l, r]`

1. Return `prefix_sum(r) - prefix_sum(l-1)`

## 6.5. Count Inversions

**Input:** Array `arr[0..n-1]`

1. Compress coordinates (if needed): map values to `1..m`
2. Initialize BIT of size `m`
3. Set `inv_count = 0`
4. For `i = 0` to `n-1`:
  - `inv_count += total_elements_seen - prefix_sum(arr[i])`
  - `point_update(arr[i], 1)`
5. Return `inv_count`

## 6.6. Range Update, Point Query (Difference BIT)

**Input:** Range `[l, r]`, delta `delta`

- Update: `point_update(l, delta), point_update(r+1, -delta)`
- Query: `prefix_sum(i)` gives the value at `arr[i]`

## 6.7. Range Update, Range Query (Two BITs)

**Input:** Range `[l, r]`, delta `delta`

- Update:

- `point_update(B1, l, delta)`
- `point_update(B1, r+1, -delta)`
- `point_update(B2, l, delta * (l-1))`
- `point_update(B2, r+1, -delta * r)`

- Query prefix sum:

- `prefix_sum(B1, i) * i - prefix_sum(B2, i)`

## 6.8. 2D BIT

**Input:** Grid of size  $n \times m$

- Update: For  $i = x$  to  $n$  with  $i += \text{lsb}(i)$ , for  $j = y$  to  $m$  with  $j += \text{lsb}(j)$ : `bit[i][j] += delta`
- Query: For  $i = x$  down to 1 with  $i -= \text{lsb}(i)$ , for  $j = y$  down to 1 with  $j -= \text{lsb}(j)$ : `sum += bit[i][j]`

## 7. Dry Run

Problem: Build BIT, query prefix sum, point update

**Original Array (1-indexed):** `arr = [3, 2, -1, 6, 5, 4, -3, 7]` ( $n = 8$ )

### Step 1: Build BIT

| i | arr[i] | bit[i] before  | Add to $j = i + \text{lsb}(i)$ | bit after this step  |
|---|--------|----------------|--------------------------------|----------------------|
| 1 | 3      | bit[1]=3       | $j=2$ : bit[2]+=3              | bit[1]=3, bit[2]=3   |
| 2 | 2      | bit[2]=3+2=5   | $j=4$ : bit[4]+=5              | bit[2]=5, bit[4]=5   |
| 3 | -1     | bit[3]=-1      | $j=4$ : bit[4]+=-1             | bit[3]=-1, bit[4]=4  |
| 4 | 6      | bit[4]=4+6=10  | $j=8$ : bit[8]+=10             | bit[4]=10, bit[8]=10 |
| 5 | 5      | bit[5]=5       | $j=6$ : bit[6]+=5              | bit[5]=5, bit[6]=5   |
| 6 | 4      | bit[6]=5+4=9   | $j=8$ : bit[8]+=9              | bit[6]=9, bit[8]=19  |
| 7 | -3     | bit[7]=-3      | $j=8$ : bit[8]+=-3             | bit[7]=-3, bit[8]=16 |
| 8 | 7      | bit[8]=16+7=23 | $j=16 > n$ , stop              | bit[8]=23            |

**Final BIT Array:**

| Index | 1 | 2 | 3  | 4  | 5 | 6 | 7  | 8  |
|-------|---|---|----|----|---|---|----|----|
| bit   | 3 | 5 | -1 | 10 | 5 | 9 | -3 | 23 |

### Step 2: Prefix Sum Query — sum up to index 6

```
i = 6 (110)
sum = 0
bit[6] = 9 → sum = 9
i = 6 - 2 = 4 (100)
bit[4] = 10 → sum = 19
i = 4 - 4 = 0
Stop. Return 19.
```

**Verification:**  $\text{arr}[1] + \text{arr}[2] + \text{arr}[3] + \text{arr}[4] + \text{arr}[5] + \text{arr}[6] = 3 + 2 + (-1) + 6 + 5 + 4 = 19 \checkmark$

### Step 3: Range Sum Query — sum of [3, 7]

```
prefix_sum(7) - prefix_sum(2)
```

**prefix\_sum(7):**

```
i = 7 (111)
bit[7] = -3 → sum = -3
i = 7 - 1 = 6 (110)
bit[6] = 9 → sum = 6
i = 6 - 2 = 4 (100)
bit[4] = 10 → sum = 16
i = 0. Stop. Return 16.
```

**prefix\_sum(2):**

```
i = 2 (010)
bit[2] = 5 → sum = 5
i = 0. Stop. Return 5.
```

**Result:**  $16 - 5 = 11$

**Verification:**  $(-1) + 6 + 5 + 4 + (-3) = 11 \checkmark$

### Step 4: Point Update — add 10 to index 3

```
i = 3 (011)
bit[3] += 10 → bit[3] = -1 + 10 = 9
i = 3 + 1 = 4 (100)
bit[4] += 10 → bit[4] = 10 + 10 = 20
i = 4 + 4 = 8 (1000)
```

```
bit[8] += 10 → bit[8] = 23 + 10 = 33
i = 16 > 8. Stop.
```

### Updated BIT:

| Index | 1 | 2 | 3 | 4  | 5 | 6 | 7  | 8  |
|-------|---|---|---|----|---|---|----|----|
| bit   | 3 | 5 | 9 | 20 | 5 | 9 | -3 | 33 |

**Verification:**  $\text{prefix\_sum}(3) = \text{bit}[3] + \text{bit}[2] = 9 + 5 = 14$ . Original sum was  $3+2+(-1)=4$ , now  $3+2+9=14$ . ✓

## 8. C++ Implementation

### Basic Fenwick Tree

```
#include <bits/stdc++.h>
using namespace std;

class FenwickTree {
private:
    int n;
    vector<long long> bit;

public:
    // Constructor: initialize BIT of size n (1-indexed)
    FenwickTree(int size) {
        n = size;
        bit.assign(n + 1, 0);
    }

    // Build BIT from an array (1-indexed)
    FenwickTree(const vector<long long>& arr) {
        n = arr.size() - 1; // arr[0] is dummy
        bit = arr;
        for (int i = 1; i <= n; i++) {
            int j = i + (i & -i);
            if (j <= n) {
                bit[j] += bit[i];
            }
        }
    }

    // Point update: add delta at index i (1-indexed)
    void add(int i, long long delta) {
        while (i <= n) {
            bit[i] += delta;
            i += i & -i;
        }
    }
}
```

```

// Prefix sum: sum of arr[1..i]
long long sum(int i) {
    long long result = 0;
    while (i > 0) {
        result += bit[i];
        i -= i & -i;
    }
    return result;
}

// Range sum: sum of arr[1..r]
long long rangeSum(int l, int r) {
    if (l > r) return 0;
    return sum(r) - sum(l - 1);
}

// Get value at index i (using difference of prefix sums)
long long pointValue(int i) {
    return rangeSum(i, i);
}

// Find smallest index such that prefix sum >= target
// Useful for order statistics (kth element)
int lowerBound(long long target) {
    int idx = 0;
    long long currentSum = 0;
    // Start from highest power of 2 <= n
    int bitMask = 1;
    while (bitMask <= n) bitMask <= 1;
    bitMask >>= 1;

    for (; bitMask > 0; bitMask >>= 1) {
        int next = idx + bitMask;
        if (next <= n && currentSum + bit[next] < target) {
            currentSum += bit[next];
            idx = next;
        }
    }
    return idx + 1;
}

};

// --- Usage Example ---
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    // 1-indexed array (index 0 is dummy)
    vector<long long> arr = {0, 3, 2, -1, 6, 5, 4, -3, 7};
    int n = 8;

    FenwickTree ft(arr);

```

```

// Prefix sum
cout << "Sum up to 6: " << ft.sum(6) << "\n";          // 19

// Range sum
cout << "Sum [3,7]: " << ft.rangeSum(3, 7) << "\n";      // 11

// Point update
ft.add(3, 10);
cout << "After adding 10 at index 3:\n";
cout << "Sum up to 6: " << ft.sum(6) << "\n";          // 29
cout << "Sum [3,7]: " << ft.rangeSum(3, 7) << "\n";      // 21

// Point value
cout << "Value at index 3: " << ft.pointValue(3) << "\n"; // 9

// Lower bound (first index where prefix sum >= target)
cout << "First index with prefix sum >= 10: " << ft.lowerBound(10) << "\n"; //
3

return 0;
}

```

## Range Update + Range Query (Two BITs)

```

#include <bits/stdc++.h>
using namespace std;

class RangeUpdateRangeQueryBIT {
private:
    int n;
    vector<long long> B1, B2; // Two BITs

    void add(vector<long long>& bit, int i, long long delta) {
        while (i <= n) {
            bit[i] += delta;
            i += i & -i;
        }
    }

    long long sum(const vector<long long>& bit, int i) {
        long long result = 0;
        while (i > 0) {
            result += bit[i];
            i -= i & -i;
        }
        return result;
    }

public:
    RangeUpdateRangeQueryBIT(int size) {
        n = size;
    }

```

```

        B1.assign(n + 1, 0);
        B2.assign(n + 1, 0);
    }

    // Range update: add delta to all elements in [l, r]
    void rangeAdd(int l, int r, long long delta) {
        add(B1, l, delta);
        add(B1, r + 1, -delta);
        add(B2, l, delta * (1 - 1));
        add(B2, r + 1, -delta * r);
    }

    // Prefix sum: sum of arr[1..i]
    long long prefixSum(int i) {
        return sum(B1, i) * i - sum(B2, i);
    }

    // Range sum: sum of arr[1..r]
    long long rangeSum(int l, int r) {
        return prefixSum(r) - prefixSum(l - 1);
    }

    // Point value at index i
    long long pointValue(int i) {
        return prefixSum(i) - prefixSum(i - 1);
    }
};

// --- Usage Example ---
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 8;
    RangeUpdateRangeQueryBIT bit(n);

    // Initial array: [0, 0, 0, 0, 0, 0, 0, 0]

    // Add 5 to range [2, 5]
    bit.rangeAdd(2, 5, 5);
    cout << "After adding 5 to [2,5]:\n";
    cout << "Sum [1,8]: " << bit.rangeSum(1, 8) << "\n"; // 20

    // Add 3 to range [3, 7]
    bit.rangeAdd(3, 7, 3);
    cout << "After adding 3 to [3,7]:\n";
    cout << "Sum [1,8]: " << bit.rangeSum(1, 8) << "\n"; // 35

    // Query point values
    cout << "Value at index 2: " << bit.pointValue(2) << "\n"; // 5
    cout << "Value at index 4: " << bit.pointValue(4) << "\n"; // 8
    cout << "Value at index 8: " << bit.pointValue(8) << "\n"; // 0

    // Query range sum

```

```

    cout << "Sum [3,6]: " << bit.rangeSum(3, 6) << "\n"; // 8+8+8+5 = 29

    return 0;
}

```

## 2D Fenwick Tree

```

#include <bits/stdc++.h>
using namespace std;

class FenwickTree2D {
private:
    int n, m;
    vector<vector<long long>> bit;

public:
    FenwickTree2D(int rows, int cols) {
        n = rows;
        m = cols;
        bit.assign(n + 1, vector<long long>(m + 1, 0));
    }

    // Point update: add delta at (x, y) (1-indexed)
    void add(int x, int y, long long delta) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                bit[i][j] += delta;
            }
        }
    }

    // Prefix sum: sum of rectangle [1..x] × [1..y]
    long long sum(int x, int y) {
        long long result = 0;
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                result += bit[i][j];
            }
        }
        return result;
    }

    // Range sum: sum of rectangle [x1..x2] × [y1..y2]
    long long rangeSum(int x1, int y1, int x2, int y2) {
        return sum(x2, y2)
            - sum(x1 - 1, y2)
            - sum(x2, y1 - 1)
            + sum(x1 - 1, y1 - 1);
    }
};

```

```
// --- Usage Example ---
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n = 5, m = 5;
    FenwickTree2D ft(n, m);

    // Add values to the grid
    ft.add(2, 3, 7);
    ft.add(4, 2, 5);
    ft.add(3, 4, 3);

    // Query prefix sum up to (4, 4)
    cout << "Sum of rectangle [1..4]x[1..4]: " << ft.sum(4, 4) << "\n"; // 15

    // Query range sum of rectangle [2..4]x[2..4]
    cout << "Sum of rectangle [2..4]x[2..4]: " << ft.rangeSum(2, 2, 4, 4) << "\n";
    // 15

    // Query range sum of rectangle [1..2]x[1..3]
    cout << "Sum of rectangle [1..2]x[1..3]: " << ft.rangeSum(1, 1, 2, 3) << "\n";
    // 7

    return 0;
}
```

## Count Inversions with BIT

```
#include <bits/stdc++.h>
using namespace std;

class FenwickTree {
    // ... (same as basic FenwickTree above) ...
};

long long countInversions(vector<int>& arr) {
    int n = arr.size();

    // Coordinate compression
    vector<int> sorted = arr;
    sort(sorted.begin(), sorted.end());
    sorted.erase(unique(sorted.begin(), sorted.end()), sorted.end());

    unordered_map<int, int> compress;
    for (int i = 0; i < (int)sorted.size(); i++) {
        compress[sorted[i]] = i + 1; // 1-indexed
    }

    int m = sorted.size();
    FenwickTree ft(m);
```

```

long long invCount = 0;

for (int i = 0; i < n; i++) {
    int rank = compress[arr[i]];
    // Number of elements greater than current seen so far
    // = total seen - prefix sum up to current rank
    invCount += ft.sum(m) - ft.sum(rank);
    ft.add(rank, 1);
}

return invCount;
}

// --- Usage Example ---
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    vector<int> arr = {8, 4, 2, 1};
    cout << "Inversion count: " << countInversions(arr) << "\n"; // 6

    arr = {3, 1, 2};
    cout << "Inversion count: " << countInversions(arr) << "\n"; // 2

    arr = {1, 2, 3, 4, 5};
    cout << "Inversion count: " << countInversions(arr) << "\n"; // 0

    return 0;
}

```

## 9. Python Implementation

### Basic Fenwick Tree

```

class FenwickTree:
    """Fenwick Tree / Binary Indexed Tree (1-indexed)"""

    def __init__(self, n: int):
        self.n = n
        self.bit = [0] * (n + 1)

    @classmethod
    def from_array(cls, arr: list):
        """Build BIT from a 1-indexed array (arr[0] is dummy)"""
        n = len(arr) - 1
        ft = cls(n)
        ft.bit = arr[:] # copy
        for i in range(1, n + 1):
            j = i + (i & -i)
            if j <= n:

```

```

        ft.bit[j] += ft.bit[i]
    return ft

def add(self, i: int, delta: int):
    """Add delta at index i (1-indexed)"""
    while i <= self.n:
        self.bit[i] += delta
        i += i & -i

def sum(self, i: int) -> int:
    """Prefix sum of arr[1..i]"""
    result = 0
    while i > 0:
        result += self.bit[i]
        i -= i & -i
    return result

def range_sum(self, l: int, r: int) -> int:
    """Sum of arr[l..r]"""
    if l > r:
        return 0
    return self.sum(r) - self.sum(l - 1)

def point_value(self, i: int) -> int:
    """Value at index i"""
    return self.range_sum(i, i)

def lower_bound(self, target: int) -> int:
    """Smallest index such that prefix sum >= target"""
    idx = 0
    bit_mask = 1
    while bit_mask <= self.n:
        bit_mask <<= 1
    bit_mask >>= 1

    while bit_mask:
        nxt = idx + bit_mask
        if nxt <= self.n and self.bit[nxt] < target:
            target -= self.bit[nxt]
            idx = nxt
        bit_mask >>= 1
    return idx + 1

# --- Usage Example ---
if __name__ == "__main__":
    # 1-indexed array
    arr = [0, 3, 2, -1, 6, 5, 4, -3, 7]
    ft = FenwickTree.from_array(arr)

    print("Sum up to 6:", ft.sum(6))          # 19
    print("Sum [3,7]:", ft.range_sum(3, 7))  # 11

    ft.add(3, 10)

```

```

print("After adding 10 at index 3:")
print("Sum up to 6:", ft.sum(6))          # 29
print("Sum [3,7]:", ft.range_sum(3, 7))   # 21
print("Value at index 3:", ft.point_value(3)) # 9

```

## Range Update + Range Query (Two BITs)

```

class RangeUpdateRangeQueryBIT:
    """Range Update + Range Query using two Fenwick Trees"""

    def __init__(self, n: int):
        self.n = n
        self.B1 = [0] * (n + 1)
        self.B2 = [0] * (n + 1)

    def _add(self, bit: list, i: int, delta: int):
        while i <= self.n:
            bit[i] += delta
            i += i & -i

    def _sum(self, bit: list, i: int) -> int:
        result = 0
        while i > 0:
            result += bit[i]
            i -= i & -i
        return result

    def range_add(self, l: int, r: int, delta: int):
        """Add delta to all elements in [l, r]"""
        self._add(self.B1, l, delta)
        self._add(self.B1, r + 1, -delta)
        self._add(self.B2, l, delta * (l - 1))
        self._add(self.B2, r + 1, -delta * r)

    def prefix_sum(self, i: int) -> int:
        """Sum of arr[1..i]"""
        return self._sum(self.B1, i) * i - self._sum(self.B2, i)

    def range_sum(self, l: int, r: int) -> int:
        """Sum of arr[l..r]"""
        return self.prefix_sum(r) - self.prefix_sum(l - 1)

    def point_value(self, i: int) -> int:
        """Value at index i"""
        return self.prefix_sum(i) - self.prefix_sum(i - 1)

# --- Usage Example ---
if __name__ == "__main__":
    bit = RangeUpdateRangeQueryBIT(8)

```

```

bit.range_add(2, 5, 5)
print("After adding 5 to [2,5]:")
print("Sum [1,8]:", bit.range_sum(1, 8))    # 20

bit.range_add(3, 7, 3)
print("After adding 3 to [3,7]:")
print("Sum [1,8]:", bit.range_sum(1, 8))    # 35

print("Value at index 2:", bit.point_value(2)) # 5
print("Value at index 4:", bit.point_value(4)) # 8
print("Sum [3,6]:", bit.range_sum(3, 6))    # 29

```

## 2D Fenwick Tree

```

class FenwickTree2D:
    """2D Fenwick Tree / Binary Indexed Tree"""

    def __init__(self, n: int, m: int):
        self.n = n
        self.m = m
        self.bit = [[0] * (m + 1) for _ in range(n + 1)]

    def add(self, x: int, y: int, delta: int):
        """Add delta at (x, y) (1-indexed)"""
        i = x
        while i <= self.n:
            j = y
            while j <= self.m:
                self.bit[i][j] += delta
                j += j & -j
            i += i & -i

    def sum(self, x: int, y: int) -> int:
        """Prefix sum of rectangle [1..x] × [1..y]"""
        result = 0
        i = x
        while i > 0:
            j = y
            while j > 0:
                result += self.bit[i][j]
                j -= j & -j
            i -= i & -i
        return result

    def range_sum(self, x1: int, y1: int, x2: int, y2: int) -> int:
        """Sum of rectangle [x1..x2] × [y1..y2]"""
        return (self.sum(x2, y2)
                - self.sum(x1 - 1, y2)
                - self.sum(x2, y1 - 1)
                + self.sum(x1 - 1, y1 - 1))

```

```
# --- Usage Example ---
if __name__ == "__main__":
    ft = FenwickTree2D(5, 5)

    ft.add(2, 3, 7)
    ft.add(4, 2, 5)
    ft.add(3, 4, 3)

    print("Sum [1..4]x[1..4]:", ft.sum(4, 4))          # 15
    print("Sum [2..4]x[2..4]:", ft.range_sum(2, 2, 4, 4)) # 15
    print("Sum [1..2]x[1..3]:", ft.range_sum(1, 1, 2, 3)) # 7
```

## Count Inversions with BIT

```
def count_inversions(arr: list) -> int:
    """Count inversions using BIT with coordinate compression"""
    # Coordinate compression
    sorted_arr = sorted(set(arr))
    compress = {val: idx + 1 for idx, val in enumerate(sorted_arr)}

    m = len(sorted_arr)
    ft = FenwickTree(m)
    inv_count = 0

    for val in arr:
        rank = compress[val]
        # Number of elements greater than current seen so far
        inv_count += ft.sum(m) - ft.sum(rank)
        ft.add(rank, 1)

    return inv_count

# --- Usage Example ---
if __name__ == "__main__":
    print(count_inversions([8, 4, 2, 1])) # 6
    print(count_inversions([3, 1, 2]))   # 2
    print(count_inversions([1, 2, 3, 4])) # 0
    print(count_inversions([5, 5, 5, 5])) # 0
```

## 10. Code Explanation

### Basic Fenwick Tree

#### Constructor (`FenwickTree(int size)`):

- `n = size` stores the number of elements.
- `bit.assign(n + 1, 0)` creates a BIT array of size `n+1` (index 0 is unused). Initialized to 0.

**Build from array (FenwickTree(const vector<long long>& arr)):**

- `n = arr.size() - 1` because `arr[0]` is a dummy element.
- `bit = arr` copies the array.
- For `i = 1` to `n`: add `bit[i]` to `bit[i + lsb(i)]` if within bounds. This propagates the contribution of each element to higher indices that cover it.

**Point Update (add):**

- Starting from `i`, add `delta` to `bit[i]`.
- Move to `i + lsb(i)` which is the next index whose range includes `i`.
- Continue until `i > n`.
- Each update touches  $O(\log n)$  indices.

**Prefix Sum (sum):**

- Starting from `i`, add `bit[i]` to result.
- Move to `i - lsb(i)` which removes the current range and moves to the next range.
- Continue until `i = 0`.
- Each query touches  $O(\log n)$  indices.

**Range Sum (rangeSum):**

- Uses inclusion-exclusion: `sum(r) - sum(l-1)`.
- This works because prefix sums are cumulative.

**Lower Bound (lowerBound):**

- Binary lifting on the BIT to find smallest index with prefix sum  $\geq$  target.
- Start with the highest power of  $2 \leq n$ .
- Try to jump: if the next index is within bounds and the current sum + `bit[next]`  $<$  target, take the jump.
- Return `idx + 1` (the first index where sum  $\geq$  target).

**Range Update + Range Query (Two BITs)****Mathematical Derivation:**

- We want to support `range_add(l, r, delta)` and `range_sum(l, r)`.
- Define a difference array `diff[i] = arr[i] - arr[i-1]` (with `arr[0] = 0`).
- `range_add(l, r, delta) → diff[l] += delta, diff[r+1] -= delta`.
- `arr[i] = sum(diff[1..i])`.
- `prefix_sum(i) = sum(arr[1..i]) = sum(sum(diff[1..j]) for j=1..i)`.
- This simplifies to `prefix_sum(i) = (i+1) * sum(diff[1..i]) - sum(j * diff[j] for j=1..i)`.
- So we maintain two BITs: `B1` for `diff` and `B2` for `i * diff[i]`.

**Range Update:**

- `add(B1, l, delta), add(B1, r+1, -delta)` — updates the difference array.
- `add(B2, l, delta * (l-1)), add(B2, r+1, -delta * r)` — updates the weighted difference array.

**Prefix Sum Query:**

- $\text{sum}(B1, i) * i - \text{sum}(B2, i)$  — the formula derived above.

## 2D Fenwick Tree

- Extends the 1D BIT idea to 2D.
- Each cell  $(i, j)$  stores the sum of a sub-rectangle of the original grid.
- **Update**: Nested loops. For each  $i$  touched by the x-coordinate, iterate over all  $j$  touched by the y-coordinate.
- **Query**: Nested loops. For each  $i$  in the prefix, iterate over all  $j$  in the prefix.
- Complexity:  $O(\log n \times \log m)$  per operation.

## Count Inversions

### Coordinate Compression:

- Map each value to its rank (1-indexed) in sorted order.
- $\text{compress}[\text{val}] = \text{rank}$  where  $\text{rank}$  is the position in sorted unique values.
- This reduces the value range to  $[1, m]$  where  $m \leq n$ .

### Algorithm:

- Iterate left to right through the array.
- For each element, query how many elements seen so far are greater than it.
  - $\text{ft.sum}(m)$  gives total elements seen so far.
  - $\text{ft.sum}(\text{rank})$  gives elements  $\leq$  current (including current after adding).
  - So  $\text{ft.sum}(m) - \text{ft.sum}(\text{rank})$  gives elements  $>$  current.
- Add 1 to the BIT at the current rank.
- This counts all pairs  $(i, j)$  with  $i < j$  and  $\text{arr}[i] > \text{arr}[j]$ .

## 11. Complexity Analysis

| Operation                  | Time                      | Space           | Notes                                   |
|----------------------------|---------------------------|-----------------|-----------------------------------------|
| Build ( $O(n \log n)$ )    | $O(n \log n)$             | $O(n)$          | Naive: call <b>add</b> for each element |
| Build ( $O(n)$ )           | $O(n)$                    | $O(n)$          | Using the propagation method            |
| Point Update               | $O(\log n)$               | $O(1)$ extra    |                                         |
| Prefix Sum Query           | $O(\log n)$               | $O(1)$ extra    |                                         |
| Range Sum Query            | $O(\log n)$               | $O(1)$ extra    | Two prefix sum calls                    |
| Point Value                | $O(\log n)$               | $O(1)$ extra    |                                         |
| Lower Bound (kth element)  | $O(\log n)$               | $O(1)$ extra    | Binary lifting on BIT                   |
| Count Inversions           | $O(n \log n)$             | $O(n)$          | With coordinate compression             |
| Range Update + Range Query | $O(\log n)$ per operation | $O(n)$          | Two BITs                                |
| 2D BIT Point Update        | $O(\log n \times \log m)$ | $O(n \times m)$ |                                         |

| Operation               | Time                      | Space        | Notes        |
|-------------------------|---------------------------|--------------|--------------|
| 2D BIT Prefix Sum Query | $O(\log n \times \log m)$ | $O(1)$ extra |              |
| Coordinate Compression  | $O(n \log n)$             | $O(n)$       | Sorting step |

### Best/Worst Case

- **Best case:** All operations are  $O(\log n)$ . The tree is balanced by definition.
- **Worst case:** Same as best case —  $O(\log n)$  is guaranteed, not amortized.
- **Space:**  $O(n)$  for 1D,  $O(n \times m)$  for 2D.

## 12. Common Patterns

### Pattern 1: Point Update, Range Sum Query

**How to Identify:** Problems that ask for sum of a subarray and allow updating individual elements.

**General Approach:** Standard BIT. Update at index, query prefix sums.

**Examples:**

- Range Sum Query — Mutable (LeetCode 307)
- Update and Query (CSES 1648)

### Pattern 2: Count Inversions / Greater Elements

**How to Identify:** "Count pairs  $(i, j)$  with  $i < j$  and  $a[i] > a[j]$ ", "count smaller numbers after self".

**General Approach:** Process right to left (or left to right), use BIT to count elements seen so far that are greater/smaller.

**Examples:**

- Count of Smaller Numbers After Self (LeetCode 315)
- Inversion Count (GFG)

### Pattern 3: Order Statistics / Kth Element in Multiset

**How to Identify:** "Find kth smallest element", "find median in a stream", "rank of element".

**General Approach:** Use BIT as a frequency array. Use `lower_bound` (binary lifting) to find the kth element.

**Examples:**

- Kth Smallest Element in sorted order (custom)
- Find Median from Data Stream (LeetCode 295 — can use two BITs)

### Pattern 4: Range Update, Point Query

**How to Identify:** "Add value to range  $[l, r]$ ", "get value at index  $i$ ".

**General Approach:** Use a single BIT as a difference array. `add(1, delta)`, `add(r+1, -delta)`, then `sum(i)` gives the value at `i`.

**Examples:**

- Difference Array variant
- Range Addition (LeetCode 370)

### Pattern 5: Range Update, Range Query

**How to Identify:** "Add value to range `[l, r]`", "get sum of range `[l, r]`".

**General Approach:** Two BITs using the difference array mathematical derivation.

**Examples:**

- Range Update and Range Query (custom)
- Segment Tree alternative

### Pattern 6: 2D Range Sum Query

**How to Identify:** "Sum of submatrix", "grid point updates", "2D prefix sum with updates".

**General Approach:** 2D BIT with nested loops.

**Examples:**

- Range Sum Query 2D — Mutable (LeetCode 308)
- Forest Queries II (CSES 1652)

### Pattern 7: Coordinate Compression + BIT

**How to Identify:** Large value range (up to  $10^9$ ) but few elements ( $n \leq 10^5$ ).

**General Approach:** Compress values to ranks, then use BIT on compressed values.

**Examples:**

- Count of Smaller Numbers After Self (LeetCode 315)
- Count inversions with large values

### Pattern 8: Longest Increasing Subsequence (LIS) with BIT

**How to Identify:** "Length of LIS", "number of LIS", "LIS in  $O(n \log n)$ ".

**General Approach:** Use BIT to store the best LIS length ending at each value. For each element, query the best length among smaller values.

**Examples:**

- Longest Increasing Subsequence (LeetCode 300 — BIT variant)
- Number of Longest Increasing Subsequence (LeetCode 673)

## 13. Common Mistakes

| Mistake                                               | Explanation                                                                                                                        | Fix                                                                                                                                        |
|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>0-based indexing</b>                               | BIT works naturally with 1-based indexing. Using 0 causes infinite loops.                                                          | Add 1 to all indices, or use <code>i += 1</code> when working with 0-based input.                                                          |
| <b>Forgetting <code>i &amp; -i</code> parentheses</b> | <code>i &amp; -i</code> has lower precedence than <code>+=</code> in some languages.                                               | Use <code>i += (i &amp; -i)</code> or <code>i += i &amp; -i</code> (works in C++ due to precedence, but be explicit).                      |
| <b>Integer overflow</b>                               | BIT stores sums that can grow large. Use <code>long long</code> in C++ / <code>int</code> in Python may overflow for large inputs. | Use <code>long long</code> in C++, <code>int</code> is fine in Python (unbounded).                                                         |
| <b>BIT size off by one</b>                            | Creating BIT of size <code>n</code> but trying to access index <code>n</code> .                                                    | BIT should be size <code>n+1</code> (indices 1..n).                                                                                        |
| <b>Not handling empty BIT</b>                         | <code>lower_bound</code> on empty BIT (all zeros) returns 1 instead of 0 or -1.                                                    | Check if total sum is 0 before calling <code>lower_bound</code> , or handle separately.                                                    |
| <b>Wrong range update formula</b>                     | Using the wrong formula for range update + range query.                                                                            | Remember: <code>prefix_sum(i) = sum(B1, i) * i - sum(B2, i)</code> .                                                                       |
| <b>Not compressing coordinates</b>                    | Using BIT directly on values up to $10^9$ .                                                                                        | Always compress if values are large relative to <code>n</code> .                                                                           |
| <b>Duplicate values in inversion count</b>            | Counting equal elements as inversions.                                                                                             | Be careful: <code>sum(m) - sum(rank)</code> counts elements strictly greater. If you want $\geq$ , use <code>sum(m) - sum(rank-1)</code> . |
| <b>2D BIT memory</b>                                  | Creating a 2D BIT of size $10^5 \times 10^5$ .                                                                                     | 2D BIT is only feasible when both dimensions are small ( $\leq 10^3$ ). Use offline methods or 2D segment tree for larger.                 |
| <b>Not resetting BIT between test cases</b>           | Reusing BIT without resetting.                                                                                                     | Reinitialize the BIT for each test case.                                                                                                   |
| <b>Using BIT for range minimum/maximum queries</b>    | BIT only works for invertible operations (sum, xor, product).                                                                      | Use Segment Tree for min/max.                                                                                                              |

## 14. Edge Cases

| Edge Case                               | What Happens                  | How to Handle                            |
|-----------------------------------------|-------------------------------|------------------------------------------|
| <b>Empty array (<code>n = 0</code>)</b> | BIT of size 0 is meaningless. | Handle separately, return 0 for queries. |

| Edge Case                                         | What Happens                                               | How to Handle                                                                     |
|---------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Single element (n = 1)</b>                     | BIT works normally. Only bit[1] stores the value.          | Test with n=1 to ensure index 1 is handled.                                       |
| <b>All zeros</b>                                  | Sums are all 0.                                            | Works fine. <code>lower_bound</code> with target > 0 returns n+1 (no such index). |
| <b>All equal elements</b>                         | Inversion count is 0.                                      | Works fine.                                                                       |
| <b>Already sorted array</b>                       | Inversion count is 0. BIT works fine for other operations. | Test to ensure no off-by-one errors.                                              |
| <b>Reverse sorted array</b>                       | Maximum inversion count: $n*(n-1)/2$ .                     | Ensure <code>long long</code> is used for large n.                                |
| <b>Large values (up to <math>10^9</math>)</b>     | Cannot index BIT directly.                                 | Use coordinate compression.                                                       |
| <b>Negative values</b>                            | BIT works with negative values.                            | Coordinate compression needs to handle negatives. Use sorting.                    |
| <b>Very large n (<math>10^6</math>)</b>           | BIT of size $10^6$ is fine ( $O(n)$ memory).               | Ensure <code>long long</code> for sums. $O(n)$ memory is acceptable.              |
| <b>Updates at index 0</b>                         | Using 0-based indexing by mistake causes infinite loop.    | Always use 1-based internally.                                                    |
| <b>Range update with l = 1</b>                    | Need to update B1 and B2 at index 1.                       | <code>delta * (1-1) = 0</code> , so B2 update at l is 0. Works fine.              |
| <b>Range update with r = n</b>                    | Need to update at r+1 which is n+1 (out of bounds).        | Only update if $r+1 \leq n$ .                                                     |
| <b>2D BIT with single row/column</b>              | Degrades to 1D BIT.                                        | Works fine.                                                                       |
| <b>Duplicate values in coordinate compression</b> | Need to assign same rank to duplicate values.              | Use <code>sort + unique</code> or <code>sorted(set(arr))</code> .                 |

## 15. Variations

### 15.1. Fenwick Tree for Range Minimum Query (RMQ)

**What changes:** Instead of storing sum, each node stores the minimum of its range.

**When used:** When you need range minimum with point updates.

**Important:** This only works for **prefix minimum** queries, not arbitrary range minimum. To get range minimum, you need to query from the right side as well, which is not straightforward. For arbitrary range min, use a Segment Tree.

**Placements/CP:** Not commonly used. Segment Tree is preferred.

### 15.2. Fenwick Tree for XOR Queries

**What changes:** Store XOR instead of sum. `bit[i] ^= delta` instead of `+=`.

**When used:** When you need range XOR queries with point updates.

**Important:** XOR is invertible (XOR is its own inverse), so BIT works perfectly.

**Placements/CP:** Occasionally useful. Same complexity as sum BIT.

### 15.3. Fenwick Tree for Product (with modular arithmetic)

**What changes:** Store product modulo M. `bit[i] = (bit[i] * delta) % M` for updates, multiply for prefix queries.

**When used:** When you need range product queries modulo M.

**Important:** Division needs modular inverse. For non-modular product, values can overflow quickly.

**Placements/CP:** Rare. Segment Tree is usually preferred.

### 15.4. 2D Fenwick Tree (already covered)

**What changes:** Nested loops for update and query.

**When used:** 2D grid with point updates and prefix sum queries.

**Important:**  $O(n^2)$  memory. Only feasible for  $n, m \leq 10^3$ .

**Placements/CP:** Important for specific problems. Know the implementation.

### 15.5. 3D Fenwick Tree

**What changes:** Three nested loops.

**When used:** 3D grid with point updates.

**Important:**  $O(n^3)$  memory. Only feasible for very small dimensions.

**Placements/CP:** Rare. Mostly theoretical.

### 15.6. BIT with Range Update and Range Query (already covered)

**What changes:** Two BITs instead of one.

**When used:** Range add + range sum queries.

**Important:** More complex than a single BIT, but simpler than a Segment Tree with lazy propagation.

**Placements/CP:** Important. Know the derivation and implementation.

### 15.7. BIT for Order Statistics (Kth Order Statistic)

**What changes:** BIT stores frequencies. `lower_bound` finds the kth element.

**When used:** "Find kth smallest element in a multiset", "median in a stream".

**Important:** Requires binary lifting on BIT.  $O(\log n)$  per query.

**Placements/CP:** Very important. Common in problems.

## 16. Related Algorithms/Data Structures

### Fenwick Tree vs Segment Tree

| Aspect                     | Fenwick Tree                   | Segment Tree                      |
|----------------------------|--------------------------------|-----------------------------------|
| Operations                 | Sum, XOR, product (invertible) | Sum, min, max, GCD, AND, OR, etc. |
| Complexity                 | $O(\log n)$                    | $O(\log n)$                       |
| Memory                     | $O(n)$                         | $O(4n)$                           |
| Code complexity            | Simple (~10 lines)             | More complex (~50 lines)          |
| Range update + range query | Two BITs needed                | Lazy propagation                  |
| 2D extension               | Nested loops                   | Quad tree                         |
| Lower bound                | Yes (binary lifting)           | Yes (walk the tree)               |

#### Choose Fenwick Tree when:

- You only need sum/XOR/product operations
- You want the simplest, fastest code
- Memory is a constraint

#### Choose Segment Tree when:

- You need non-invertible operations (min, max, GCD)
- You need lazy propagation extensively
- You need to combine multiple operations (sum + min)

### Prefix Sum Array vs Fenwick Tree

| Aspect          | Prefix Sum Array | Fenwick Tree |
|-----------------|------------------|--------------|
| Build           | $O(n)$           | $O(n)$       |
| Range sum query | $O(1)$           | $O(\log n)$  |
| Point update    | $O(n)$           | $O(\log n)$  |
| Memory          | $O(n)$           | $O(n)$       |

**Choose Prefix Sum Array when:** The array is static (no updates).

**Choose Fenwick Tree when:** You need both queries and updates.

### Difference Array vs Fenwick Tree

| Aspect          | Difference Array  | Fenwick Tree |
|-----------------|-------------------|--------------|
| Range update    | $O(1)$            | $O(\log n)$  |
| Point query     | $O(n)$ worst case | $O(\log n)$  |
| Range sum query | $O(n)$            | $O(\log n)$  |

**Choose Difference Array when:** Only range updates and point queries are needed, and  $O(n)$  per query is acceptable.

**Choose Fenwick Tree when:** You need efficient range sum queries as well.

## Ordered Set (Balanced BST) vs Fenwick Tree

| Aspect        | Ordered Set (e.g., <code>std::set</code> + order statistics) | Fenwick Tree               |
|---------------|--------------------------------------------------------------|----------------------------|
| Insert/delete | $O(\log n)$                                                  | Not supported (fixed size) |
| Kth element   | $O(\log n)$                                                  | $O(\log n)$                |
| Rank query    | $O(\log n)$                                                  | $O(\log n)$                |
| Memory        | $O(n)$                                                       | $O(n)$                     |
| Rebalancing   | Automatic                                                    | Not needed (static)        |

**Choose Ordered Set when:** You need dynamic insert/delete.

**Choose Fenwick Tree when:** The set size is fixed and you only need to update frequencies.

## 17. Practice Problems

### Easy

#### 1. Range Sum Query — Mutable

- **Platform:** LeetCode 307
- **Main Idea:** Standard point update, range sum query.
- **Difficulty:** Easy

#### 2. Forest Queries

- **Platform:** CSES 1652
- **Main Idea:** 2D prefix sum query (static — can use 2D prefix sum, but 2D BIT works).
- **Difficulty:** Easy

### Medium

#### 1. Count of Smaller Numbers After Self

- **Platform:** LeetCode 315

- **Main Idea:** Process right to left, use BIT with coordinate compression.
- **Difficulty:** Medium

## 2. Inversion Count

- **Platform:** GFG, SPOJ
- **Main Idea:** Count inversions using BIT with coordinate compression.
- **Difficulty:** Medium

## 3. Range Update and Range Query

- **Platform:** CSES 1651 (Range Update Queries)
- **Main Idea:** Range update + point query, or range update + range query with two BITs.
- **Difficulty:** Medium

Hard

## 1. Count of Range Sum

- **Platform:** LeetCode 327
- **Main Idea:** Prefix sums + BIT. Count number of pairs  $(i, j)$  such that  $\text{prefixSum}[j] - \text{prefixSum}[i]$  is in  $[\text{lower}, \text{upper}]$ .
- **Difficulty:** Hard

## 2. Reverse Pairs

- **Platform:** LeetCode 493
- **Main Idea:** Count pairs  $(i, j)$  with  $i < j$  and  $\text{arr}[i] > 2 \cdot \text{arr}[j]$ . BIT with coordinate compression.
- **Difficulty:** Hard

## 3. Range Sum Query 2D — Mutable

- **Platform:** LeetCode 308
- **Main Idea:** 2D BIT for point updates and range sum queries on a matrix.
- **Difficulty:** Hard

# 18. Interview Explanation

"A Fenwick Tree, or Binary Indexed Tree, is a data structure that maintains an array and supports prefix sum queries and point updates, both in  $O(\log n)$  time.

The core idea is based on the binary representation of indices. Each index  $i$  in the BIT stores the sum of a range of the original array — specifically, the range from  $i - \text{LSB}(i) + 1$  to  $i$ , where  $\text{LSB}$  is the lowest set bit. This range length is always a power of 2.

To get a prefix sum up to index  $i$ , we start at  $i$  and keep subtracting the  $\text{LSB}$ , accumulating the values at each index. This effectively breaks the prefix into  $O(\log n)$  disjoint ranges that together cover  $[1, i]$ .

To update a value at index  $i$ , we start at  $i$  and keep adding the LSB, updating all indices whose ranges include  $i$ . There are  $O(\log n)$  such indices.

The key operation is  $i \& -i$ , which isolates the lowest set bit. This lets us traverse the tree efficiently.

I can also use a BIT for range sum queries by computing  $\text{prefixSum}(r) - \text{prefixSum}(l-1)$ . For range updates and range queries, I use two BITs. For 2D problems, I extend the idea with nested loops.

Compared to a Segment Tree, the BIT is simpler, uses less memory ( $O(n)$  vs  $O(4n)$ ), and is faster in practice, but it only works for invertible operations like sum, XOR, or product."

## 19. Revision Notes

- **Key Idea:** BIT stores partial sums in ranges determined by  $i \& -i$  (lowest set bit).
- **1-Based Indexing:** Always use 1-based internally.  $\text{index} = \text{arr\_index} + 1$ .
- **LSB:**  $i \& -i$  isolates the lowest set bit.
- **Prefix Sum:**  $\text{while}(i > 0) \text{ sum} += \text{bit}[i], i -= i \& -i$ .
- **Point Update:**  $\text{while}(i \leq n) \text{ bit}[i] += \text{delta}, i += i \& -i$ .
- **Range Sum:**  $\text{sum}(r) - \text{sum}(l-1)$ .
- **Build  $O(n)$ :** For  $i = 1..n$ ,  $\text{bit}[i] += \text{arr}[i]$ , then  $j = i + \text{lsb}(i)$ , if  $j \leq n$ ,  $\text{bit}[j] += \text{bit}[i]$ .
- **Coordinate Compression:** Sort unique values, map to ranks  $1..m$ .
- **Count Inversions:** Process left to right,  $\text{inv} += \text{totalSeen} - \text{sum}(\text{rank})$ ,  $\text{add}(\text{rank}, 1)$ .
- **Range Update + Range Query:** Two BITs —  $B1$  for diff,  $B2$  for  $i * \text{diff}[i]$ . Formula:  $\text{prefixSum}(i) = \text{sum}(B1, i) * i - \text{sum}(B2, i)$ .
- **2D BIT:** Nested loops over  $x$  and  $y$  with LSB.
- **Lower Bound:** Binary lifting on BIT to find  $k$ th element.
- **Common Traps:**
  - Off-by-one: BIT is 1-indexed.
  - Integer overflow: Use `long long` in C++.
  - BIT only works for invertible operations (sum, XOR, product).
  - 2D BIT memory is  $O(n^2)$  — use only for small dimensions.
  - For range update + range query, remember the formula correctly.

## 20. Final Cheat Sheet

### Fenwick Tree (Binary Indexed Tree)

| Aspect          | Details                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------|
| When to use     | Prefix/range sum queries + point updates, inversion count, order statistics, 2D prefix sums, range update + range query |
| Main operations | $\text{add}(i, \text{delta})$ — point update; $\text{sum}(i)$ — prefix sum; $\text{rangeSum}(l, r)$ — range sum         |
| Complexity      | $O(\log n)$ per operation, $O(n)$ space                                                                                 |
| Key code        | $i \& -i$ (LSB), $i += i \& -i$ (update), $i -= i \& -i$ (query)                                                        |

| Aspect                            | Details                                                                                                    |
|-----------------------------------|------------------------------------------------------------------------------------------------------------|
| <b>Build <math>O(n)</math></b>    | <code>for i=1..n: bit[i] += arr[i]; j = i + (i&amp;-i); if j ≤ n: bit[j] += bit[i]</code>                  |
| <b>Range Update + Range Query</b> | Two BITs: <code>prefixSum(i) = sum(B1, i) * i - sum(B2, i)</code>                                          |
| <b>2D BIT</b>                     | Nested loops: <code>for(i=x; i≤n; i+=i&amp;-i) for(j=y; j≤m; j+=j&amp;-j) bit[i][j] += delta</code>        |
| <b>Count Inversions</b>           | Compress values → BIT on ranks → for each element: <code>inv += totalSeen - sum(rank), add(rank, 1)</code> |
| <b>Important edge cases</b>       | $n=0$ , $n=1$ , all zeros, duplicates, large values (compress), negative values (compress)                 |
| <b>Common mistakes</b>            | 0-based indexing, integer overflow, wrong range update formula, using BIT for min/max                      |
| <b>C++ template</b>               | <code>FenwickTree(int n), void add(int i, T delta), T sum(int i), T rangeSum(int l, int r)</code>          |
| <b>Python template</b>            | <code>FenwickTree(n), add(i, delta), sum(i), range_sum(l, r)</code>                                        |